

ASP.NET Core Security

Transport Layer Security, HTTPS, and Access Enforcement

1. HTTP vs. HTTPS

FORMAL DEFINITION: HTTP

Hypertext Transfer Protocol (HTTP) is an application-layer protocol for transmitting hypermedia documents. It operates over TCP/IP and is intrinsically stateless and unencrypted, meaning data is transmitted in plain text.

FORMAL DEFINITION: HTTPS

Hypertext Transfer Protocol Secure (HTTPS) is an extension of HTTP. It uses Transport Layer Security (TLS) to encrypt the data packets exchanged between the client and the server, providing confidentiality, integrity, and authentication.

The Conceptual Difference: Imagine HTTP as writing your username and password on a physical postcard. Anyone processing that mail (routers, ISPs, public Wi-Fi networks) can read it (a Man-in-the-Middle attack). HTTPS takes that postcard and places it inside an indestructible, locked steel box. Only the intended server holds the cryptographic key required to unlock and read the contents.

2. SSL/TLS Certificates

FORMAL DEFINITION: TLS CERTIFICATE

A **Transport Layer Security (TLS) Certificate** is an X.509 digital data structure issued by a trusted Certificate Authority (CA). It cryptographically binds a public key to an organization's identity (domain name), allowing clients to authenticate the server and negotiate a secure symmetric encryption session.

The Conceptual Difference: If HTTPS is the locked box, the Certificate is the Digital ID Card. When a browser connects, the server presents its certificate. The browser verifies this certificate against its internal list of trusted CAs (like DigiCert or Let's Encrypt). This guarantees the browser is talking to *yourwebsite.com* and not a malicious imposter.

3. HTTPS Redirection

FORMAL DEFINITION: HTTPS REDIRECTION

A server-side configuration that intercepts requests made to the insecure HTTP port (80) and responds with an HTTP 307 (Temporary Redirect) or 308 (Permanent Redirect) status code. This response includes a `Location` header directing the client to re-issue the request to the secure HTTPS port (443).

The Conceptual Difference: Users rarely type `https://` manually. When a user types a naked domain, browsers often default to HTTP. Redirection is the server's way of acting as a friendly bouncer, refusing the insecure connection and telling the browser to try again using the secure protocol.

4. HTTP Strict Transport Security (HSTS)

FORMAL DEFINITION: HSTS

HSTS (RFC 6797) is a web security policy mechanism communicated via the `Strict-Transport-Security` HTTP response header. It instructs the user agent (browser) that it must *only* interact with the server using secure HTTPS connections for a specified period (`max-age`), and never via the insecure HTTP protocol.

The Conceptual Difference: Redirection has a flaw: the very first request still travels in plain text, leaving a brief window for hackers to intercept and hijack the session. HSTS patches this. Once a browser receives an HSTS header, it caches that rule. For the next year (or specified `max-age`), the browser will *internally* force all navigation to that site to use HTTPS before the network request ever leaves the computer. It is the ultimate point of no return for security.

Security Layers Comparison

Feature	Behavior	Vulnerability Profile
Plain HTTP	Sends data in clear text.	100% vulnerable to interception (Man-in-the-Middle).
UseHttpsRedirection()	Catches HTTP traffic and redirects to HTTPS.	The <i>first</i> request is still sent in clear text (vulnerable to downgrade attacks).
UseHsts()	Forces the browser to upgrade all future requests internally.	Maximum security. No clear text ever leaves the machine after the initial policy is cached.

Implementation in ASP.NET Core

```
/*
 * =====
 * ASP.NET CORE SECURITY: TRANSPORT LAYER (HTTPS & HSTS)
 * =====
 */
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

// 1. HSTS: The Vault Door
// Sends a header telling the browser to NEVER attempt an HTTP
// connection to this domain again.
// RULE: Never use in Development, as it permanently caches in browsers
// and will break local HTTP testing!
if (!app.Environment.IsDevelopment())
{
    // Enforces HTTPS for the next 30 days (default max-age)
    app.UseHsts();
}

// 2. Redirection: The Net
// Catches users who accidentally type `http://` and forces their
// browser to try again using `https://`. Must be placed early.
app.UseHttpsRedirection();

// ... UseAuthentication, UseAuthorization, etc.

app.Run();
```